



ASSIGNMENTS – COMPUTER GRAPHICS

Semester I – Academic Year 2026–2027

Department of Computer Science
Ho Chi Minh City University of Technology (HCMUT), VNU-HCM

Instructor: Dr. Thanh-Sach LE, MSc Tram Tran

August 29, 2026

Current version: v1.1

Version History

Version	Date	Update Details
1.0	2025-09-01	Initial version.
1.1	2026-08-29	Semester I, academic year 2026–2027; deadlines shifted to Oct/Nov 2026.

Introduction

This assignment set is designed to help students gain practical experience in modern computer graphics programming. Across both assignments, students will learn how to create and render geometric scenes using modern OpenGL or equivalent libraries across different platforms. The emphasis is on understanding and using programmable graphics pipelines with custom shaders written in GLSL (OpenGL Shading Language), rather than relying on outdated fixed-function techniques.

Depending on their preferred programming language or development environment, students may choose from a variety of rendering APIs:

- **OpenGL (C++)**: The standard cross-platform graphics API. Students are expected to use shader-based rendering with libraries such as `GLFW`, `GLEW`, or `GLM`.
- **PyOpenGL (Python)**: A Python binding for OpenGL that supports modern graphics programming. Often used with `GLFW`, `PyGLM`, or `ModernGL`.
- **WebGL (JavaScript)**: A web-based version of OpenGL ES 2.0 with shader support. Common frameworks include `Three.js` and `Babylon.js`.

By completing the two assignments, students will explore various core topics in computer graphics, including transformation matrices, 3D geometry rendering, lighting, and synthetic data generation for downstream computer vision tasks.

1 Assignment 1: 2D and 3D Object Rendering

1.1 Part 1: Drawing Basic Shapes

Students must design and implement an interactive graphical application that can render a variety of basic 2D and 3D shapes. The application must support modern rendering techniques and provide a user-friendly interface for creating and manipulating objects in the scene.

Supported shapes:

- **2D Shapes**: triangle, rectangle, pentagon, regular hexagon, circle, ellipse, trapezoid, star, arrow.
- **3D Shapes**:
 - Basic solids: cube, sphere, cylinder, cone, truncated cone, tetrahedron, torus, prism.
 - Mathematical surface defined by a user-provided function $z = f(x, y)$.
 - Imported 3D model from `.obj` or `.ply` file.

Basic requirements:

- The application must include a graphical user interface (GUI) with menus or toolbars that allow users to select which shape to draw or add.
- The application must support mouse and/or keyboard interaction to zoom, pan, and rotate the objects in the scene.
- The application must provide multiple rendering modes for each object:
 - Flat color (single uniform color for the entire object).
 - Vertex color interpolation using Gouraud shading.
 - Phong shading (with per-fragment lighting).
 - Texture mapping using external image files provided by the user.
 - Wireframe mode for visualizing only the edges of the shape.

1.2 Part 2: Visualization of Atoms and Molecules

Students must design and implement an interactive application to visualize atomic and molecular structures using modern graphics techniques.

Required features:

- The application must include a graphical user interface (GUI) with menus or toolbars that allow users to access and toggle the following features:
 - **Visualize atoms using Bohr's model:** Render electrons orbiting the nucleus in distinct shells. Electrons can be represented as small spheres revolving around the nucleus. (Reference: Wikipedia - Bohr model)
 - **Visualize simple molecules:** Render common molecules (e.g., H_2O , CO_2) where:
 - * Atoms are represented as colored spheres (e.g., red for oxygen, white for hydrogen, black for carbon).
 - * Bonds between atoms are represented as cylinders connecting the spheres.(Reference: Wikipedia - Ball-and-stick model)
 - **Optional: Animate dynamic behavior:**
 - * Electron motion along orbitals (for atomic models).
 - * Molecular vibration or rotation (for small molecules).

References:

- * Bohr Model – Wikipedia
- * Molecular Vibration – Wikipedia
- * Infrared Spectroscopy – Wikipedia

Suggestions for Implementation:

- Use scene graph structures to manage atoms and bonds as hierarchical objects.
- Use transformation matrices to animate electron motion or vibrational effects.
- Use distinct materials and lighting to emphasize atomic and bonding structures.

2 Assignment 2: Synthetic Image Generator for Detection, Segmentation, and Depth Estimation

2.1 Overview

This assignment focuses on building a synthetic image generation system capable of producing high-quality RGB images along with structured ground truth annotations for multiple vision tasks, including object detection, instance segmentation, and monocular depth estimation.

3 Deliverables

Students must submit the following two components:

1. **Report:** Submit both `.tex` and compiled `.pdf` files. The report must include:
 - Description of the application's architecture and components.
 - Detailed explanation of implemented features and user instructions.
 - Screenshots of all rendered scenes to demonstrate functionality (Assignment 2 only).
 - Link to the GitHub repository containing the full source code.
2. **Source Code Package:**
 - Full source code of the application.
 - Setup instructions, including dependencies and configuration steps.
 - Any necessary data files (e.g., textures, models) required for the program to run.

Submission Method: All materials must be uploaded to the shared assignment submission folder on the course's LMS (Learning Management System).

3.1 Timeline and Progress Evaluation

To ensure timely progress and early feedback, each assignment includes two required stages:

- **Progress Report:** Students must submit a brief report prior to the final deadline, outlining their current implementation status, screenshots (if any), and encountered challenges. This report is used for feedback and partial grading.
- **Final Submission:** Includes the complete deliverables as described above. Late submission may result in penalty unless valid justification is provided.

Assignment	Topic	Progress Tracking	Final Deadline
Assignment 1	2D/3D Shape Rendering	In-class progress demo	Oct 15, 2026
Assignment 2	Synthetic Image Generator	In-class progress demo	Nov 25, 2026

Table 2: Assignment milestones and deadlines

3.2 Grading Scheme

The course grade is based on two major assignments, each contributing 50% to the final score. Each assignment is further broken down into specific components, evaluated separately to reflect both functionality and design quality.

- **Assignment 1 (50%): 2D/3D Shape Rendering**
 - **Part 1 – Drawing Basic Shapes (25%):**
 - * Load and render basic 2D/3D shapes (e.g., cube, sphere, pyramid).
 - * Demonstrate geometric transformations (translation, rotation, scaling).
 - * Ensure code structure is clean and modular.
 - **Part 2 – Visualization of Atoms and Molecules (25%):**
 - * Model atoms as spheres and visualize simple molecules.
 - * Allow configuration of atomic types and positions.
 - * Render molecule structures with proper camera control and output screenshots.
- **Assignment 2 (50%): Synthetic Image Generator**
 - **Scene Design and Rendering (15%):**
 - * Load objects from .obj or .ply files.
 - * Configure scenes with lighting, textures, and camera settings.
 - **Ground Truth Label Generation (15%):**
 - * Export bounding boxes and segmentation masks in standard format (e.g., COCO, YOLO).
 - * Label files must match with image data and be correctly formatted.
 - **Depth Estimation Labels (10%):**
 - * Generate per-pixel depth maps or point cloud projections.
 - * Output must align with RGB image resolution.
 - **Usability and Dataset Export (10%):**
 - * Provide clear folder structure and naming convention.
 - * Include metadata files (e.g., camera pose, object positions).

Important Note

Please regularly visit the course page for **Computer Graphics (CO3059)**. This is the central hub for:

- Asking questions and discussing ideas related to assignments.
- Receiving updates on requirements, deadlines, and grading criteria.
- Sharing resources and staying connected with the class.